42.4 高级功能

> **生成模型**：Claude Opus 4.6 (anthropic/claude-opus-4-6)
> **Token 消耗**：输入 ~365k tokens，输出 ~5k tokens（本节）

基础骨架搭建完成后，本节为 MiniClaw 添加三个进阶功能：记忆系统、技能系统和 Cron 调度。这些功能参考了 OpenClaw 的对应实现（第 20-22 章），但进行了大幅简化。

42.4.1 添加记忆系统（向量搜索）

Markdown 记忆

OpenClaw 的记忆系统基于"纯 Markdown 即记忆"的理念（第 20 章）。MiniClaw 采用同样的方法：

```typescript
// src/memory/markdown.ts

import fs from "node:fs";
import path from "node:path";

const MEMORY_DIR = path.join(process.cwd(), ".miniclaw", "memory");

export class MarkdownMemory {
  constructor() {
    fs.mkdirSync(MEMORY_DIR, { recursive: true });
  }

  // 保存长期记忆
  save(key: string, content: string) {
    const filePath = path.join(MEMORY_DIR, `${key}.md`);
    fs.writeFileSync(filePath, content, "utf-8");
  }

  // 追加日志记忆
  appendLog(content: string) {
    const date = new Date().toISOString().split("T")[0];
    const filePath = path.join(MEMORY_DIR, `${date}.md`);
    const entry = `\n## ${new Date().toLocaleTimeString()}\n\n${content}\n`;
    fs.appendFileSync(filePath, entry, "utf-8");
  }

  // 加载所有记忆作为上下文
  loadAll(): string {
    const files = fs.readdirSync(MEMORY_DIR)
      .filter(f => f.endsWith(".md"))
      .sort();
    const contents = files.map(f => {
      const content = fs.readFileSync(path.join(MEMORY_DIR, f), "utf-8");
      return `# ${f}\n${content}`;
    });
    return contents.join("\n\n---\n\n");
  }
}
```

记忆工具

将记忆操作暴露为 Agent 可调用的工具：

```typescript
// src/agent/tools/memory.ts

export function createMemoryTools(memory: MarkdownMemory): ToolExecutor[] {
  return [
    {
      name: "memory_save",
      definition: {
        name: "memory_save",
        description: "Save information to long-term memory.",
        input_schema: {
          type: "object",
          properties: {
            key: { type: "string", description: "Memory key (filename)" },
            content: { type: "string", description: "Content to save" },
          },
          required: ["key", "content"],
        },
      },
      execute: async (input) => {
        memory.save(input.key as string, input.content as string);
        return `Saved to memory: ${input.key}`;
      },
    },
    {
      name: "memory_search",
      definition: {
        name: "memory_search",
        description: "Search through saved memories.",
        input_schema: {
          type: "object",
          properties: {
            query: { type: "string", description: "Search query" },
          },
          required: ["query"],
        },
      },
      execute: async (input) => {
        const all = memory.loadAll();
        // 简单的关键词搜索（生产环境应使用向量搜索）
        const query = (input.query as string).toLowerCase();
        const lines = all.split("\n");
        const matches = lines.filter(line =>
          line.toLowerCase().includes(query)
```

```
        );
        return matches.length > 0
          ? matches.slice(0, 20).join("\n")
          : "No matches found.";
      },
    },
  ];
}
```

向量搜索增强（可选）

对于需要语义搜索的场景，可以集成嵌入向量：

```typescript
// src/memory/vector.ts（可选增强）

import { OpenAI } from "openai";

export class VectorMemory {
  private openai: OpenAI;
  private vectors: Array<{ text: string; embedding: number[] }> = [];

  constructor(apiKey: string) {
    this.openai = new OpenAI({ apiKey });
  }

  async index(text: string) {
    const response = await this.openai.embeddings.create({
      model: "text-embedding-3-small",
      input: text,
    });
    this.vectors.push({
      text,
      embedding: response.data[0].embedding,
    });
  }

  async search(query: string, topK = 5) {
    const qEmbed = await this.embed(query);
    // 余弦相似度排序
    const scored = this.vectors.map(v => ({
      text: v.text,
      score: cosineSimilarity(qEmbed, v.embedding),
    }));
    scored.sort((a, b) => b.score - a.score);
    return scored.slice(0, topK);
  }
}

function cosineSimilarity(a: number[], b: number[]): number {
  let dot = 0, normA = 0, normB = 0;
  for (let i = 0; i < a.length; i++) {
    dot += a[i] * b[i];
    normA += a[i] * a[i];
    normB += b[i] * b[i];
  }
```

```
  return dot / (Math.sqrt(normA) * Math.sqrt(normB));
}
```

42.4.2 添加技能系统

OpenClaw 的技能系统（第 21 章）允许动态加载能力模块。MiniClaw 实现一个简化版：

```typescript
// src/skills/loader.ts

import fs from "node:fs";
import path from "node:path";

export type Skill = {
  name: string;
  description: string;
  systemPrompt: string;  // 注入到 Agent 的额外指令
};

const SKILLS_DIR = path.join(process.cwd(), ".miniclaw", "skills");

export function loadSkills(): Skill[] {
  if (!fs.existsSync(SKILLS_DIR)) return [];

  const skills: Skill[] = [];
  const dirs = fs.readdirSync(SKILLS_DIR, { withFileTypes: true })
    .filter(d => d.isDirectory());

  for (const dir of dirs) {
    const skillFile = path.join(SKILLS_DIR, dir.name, "SKILL.md");
    if (!fs.existsSync(skillFile)) continue;

    const content = fs.readFileSync(skillFile, "utf-8");
    skills.push({
      name: dir.name,
      description: extractDescription(content),
      systemPrompt: content,
    });
  }
  return skills;
}

function extractDescription(markdown: string): string {
  const firstLine = markdown.split("\n").find(l => l.trim() && !l.startsWith("#"));
  return firstLine?.trim() ?? "";
}
```

技能文件格式与 OpenClaw 的 `SKILL.md` 兼容：

```
<!-- .miniclaw/skills/coding/SKILL.md -->
# Coding Assistant

You are an expert programmer. Follow these rules:

1. Write clean, well-documented code
2. Use TypeScript best practices
3. Always handle errors properly
4. Prefer functional programming patterns
```

加载后的技能会被拼接到 Agent 的系统提示中：

```
const skills = loadSkills();
const skillsPrompt = skills.map(s => s.systemPrompt).join("\n\n---\n\n");
const systemPrompt = `你是一个有帮助的 AI 助手。\n\n${skillsPrompt}`;
```

42.4.3 添加 Cron 调度

参考 OpenClaw 的 Cron 系统（第 18 章），实现一个简单的定时任务调度器：

```typescript
// src/cron/scheduler.ts

import { CronJob } from "cron";  // 使用 cron 库

export type CronTask = {
  id: string;
  expression: string;  // Cron 表达式
  prompt: string;       // 发送给 Agent 的消息
  sessionKey: string;   // 目标会话
  enabled: boolean;
};

export class CronScheduler {
  private tasks = new Map<string, CronTask>();
  private jobs = new Map<string, CronJob>();
  private sendMessage: (sessionKey: string, message: string) => Promise<void>;

  constructor(sendMessage: (sessionKey: string, message: string) => Promise<void>) {
    this.sendMessage = sendMessage;
  }

  addTask(task: CronTask) {
    this.tasks.set(task.id, task);
    if (task.enabled) {
      this.startJob(task);
    }
  }

  private startJob(task: CronTask) {
    const job = new CronJob(task.expression, async () => {
      console.log(`[cron] Running task: ${task.id}`);
      try {
        await this.sendMessage(
          task.sessionKey,
          `[Scheduled task: ${task.id}] ${task.prompt}`
        );
      } catch (err) {
        console.error(`[cron] Task ${task.id} failed:`, err);
      }
    });
    job.start();
    this.jobs.set(task.id, job);
  }
```

```
  removeTask(id: string) {
    this.jobs.get(id)?.stop();
    this.jobs.delete(id);
    this.tasks.delete(id);
  }

  listTasks(): CronTask[] {
    return Array.from(this.tasks.values());
  }
}
```

配置文件驱动的定时任务：

```
// .miniclaw/cron.json
{
  "tasks": [
    {
      "id": "daily-summary",
      "expression": "0 9 * * *",
      "prompt": "请总结一下今天的日程安排和待办事项",
      "sessionKey": "telegram:123456",
      "enabled": true
    },
    {
      "id": "weather-check",
      "expression": "0 7 * * 1-5",
      "prompt": "查看今天的天气预报",
      "sessionKey": "webchat:default",
      "enabled": true
    }
  ]
}
```

Cron 工具

让 Agent 自己也能创建定时任务：

```typescript
export const cronTool: ToolExecutor = {
  name: "cron_create",
  definition: {
    name: "cron_create",
    description: "Create a scheduled task that runs at specified times.",
    input_schema: {
      type: "object",
      properties: {
        id: { type: "string", description: "Unique task ID" },
        expression: { type: "string", description: "Cron expression (e.g. '0 9 * * *')"
        prompt: { type: "string", description: "Message to process when triggered" },
      },
      required: ["id", "expression", "prompt"],
    },
  },
  execute: async (input) => {
    scheduler.addTask({
      id: input.id as string,
      expression: input.expression as string,
      prompt: input.prompt as string,
      sessionKey: "global",
      enabled: true,
    });
    return `Created cron task: ${input.id} (${input.expression})`;
  },
};
```

本节小结

1. **Markdown 记忆** 以文件为单位存储长期记忆和日志，通过关键词搜索检索。
2. **向量搜索增强** 可选集成 OpenAI Embeddings，实现语义级的记忆检索。
3. **技能系统** 通过 `SKILL.md` 文件定义额外能力，动态加载并拼接到系统提示中。
4. **Cron 调度** 使用 `cron` 库实现定时任务，支持配置文件和 Agent 工具两种创建方式。
5. 所有高级功能都遵循"可选增强"原则——核心功能不依赖它们，但加入后能显著提升助手的实用性。